

TỐI ƯU HÓA GIAO HÀNG ĐA TÁC TỬ THỜI GIAN THỰC

1st Bùi Phúc Bình
Nhóm 22
23021478@vnu.edu.vn

2nd Trần Bình Minh
Nhóm 22
22028187@vnu.edu.vn

Tóm tắt nội dung—Báo cáo trình bày bài toán tối ưu hóa giao hàng đa tác tử thời gian thực trên bản đồ lưới. Hệ thống gồm nhiều shipper cùng nhận và giao các đơn hàng có trọng lượng, mức ưu tiên và deadline khác nhau. Mục tiêu là xây dựng thuật toán phân công và điều phối shipper nhằm tối đa hóa tổng phần thưởng sau khi trừ chi phí di chuyển. Báo cáo tập trung vào các phương pháp Greedy BFS, VRP + OR-Tools, ACO và MAPD-CBS. Các thuật toán được đánh giá dựa trên net reward, tỷ lệ giao đúng hạn và thời gian chạy. Ngoài ra, báo cáo cũng phân tích cách xử lý tình huống surge và hotspot trong môi trường giao hàng động.

I. GIỚI THIỆU

Trong những năm gần đây, các hệ thống giao nhận theo thời gian thực ngày càng trở nên phổ biến trong các lĩnh vực đời sống. Đặc điểm chung của các hệ thống này là số lượng đơn hàng có thể thay đổi liên tục theo thời gian, vị trí lấy hàng và giao hàng phân bố không đồng đều trên bản đồ, trong khi đội ngũ vận chuyển phải hoạt động dưới nhiều ràng buộc như thời gian, tải trọng, số lượng đơn có thể mang theo và chi phí di chuyển. Do đó, bài toán điều phối nhiều shipper để tối đa hóa hiệu quả giao hàng là một bài toán phức tạp, có tính động cao và khó giải tối ưu trong thời gian thực.

Bài toán được xét trong báo cáo này là bài toán giao nhận nhiều tác nhân trên bản đồ dạng lưới. Mỗi shipper di chuyển từng bước trên một bản đồ $N \times N$, có thể thực hiện hành động lấy hàng tại vị trí pickup và giao hàng tại vị trí delivery. Mỗi đơn hàng có các thông tin bao gồm vị trí lấy hàng, vị trí giao hàng, trọng lượng, độ ưu tiên và hạn giao. Mục tiêu của hệ thống là tối đa hóa tổng điểm thưởng ròng, được tính từ phần thưởng giao hàng trừ đi chi phí di chuyển. Điểm thưởng phụ thuộc vào việc đơn hàng có được giao đúng hạn hay không, trong khi chi phí di chuyển phụ thuộc vào tải trọng hiện tại của shipper. Vì vậy, solver không chỉ cần tìm đường đi ngắn, mà còn phải cân bằng giữa việc nhận thêm đơn, giao đơn đang mang, tránh trễ hạn và giảm chi phí di chuyển.

Khó khăn lớn của bài toán nằm ở tính chất online và multi-agent. Trong môi trường online, không phải toàn bộ đơn hàng đều được biết trước ngay từ đầu; thay vào đó, đơn mới có thể xuất hiện trong quá trình mô phỏng. Điều này khiến một kế hoạch tối ưu ở thời điểm hiện tại có thể trở nên kém hiệu quả ở các bước sau. Bên cạnh đó, nhiều shipper cùng hoạt động trên một bản đồ nên có thể xảy ra xung đột đường đi, chen lấn tại các ô hẹp hoặc cùng hướng tới một đơn hàng. Vì vậy, thuật

toán cần vừa có khả năng lập kế hoạch dài hạn, vừa phản ứng nhanh trước các biến động mới, đồng thời phải đảm bảo thời gian chạy đủ nhỏ để phù hợp với giới hạn của hệ thống chấm điểm.

Trong báo cáo này, nhóm triển khai và so sánh bốn hướng tiếp cận chính. Phương pháp đầu tiên là Greedy BFS, đóng vai trò baseline đơn giản, trong đó mỗi shipper chọn mục tiêu tốt nhất theo heuristic cục bộ và dùng BFS để di chuyển tới mục tiêu. Phương pháp thứ hai là hướng VRP, trong đó bài toán được xem như một biến thể của Vehicle Routing Problem với nhiều shipper, nhiều điểm pickup-delivery, ràng buộc tải trọng và deadline. Trong quá trình triển khai, nhóm phát triển một solver VRP thuần Python sử dụng Hungarian assignment, cache đường đi ngắn nhất, lập kế hoạch chọn lọc và xây dựng route nhiều điểm nhằm giảm phụ thuộc vào thư viện ngoài và tăng khả năng chạy trên môi trường offline. Phương pháp thứ ba là Ant Colony Optimization, viết tắt là ACO, được sử dụng như một metaheuristic để khám phá nhiều chuỗi pickup-delivery khác nhau thay vì chỉ chọn tham lam từng bước. Phương pháp thứ tư là MAPD-CBS, kết hợp bài toán Multi-Agent Pickup and Delivery với Conflict-Based Search để xử lý xung đột đường đi giữa nhiều tác nhân.

Mục tiêu của báo cáo là phân tích nguyên lý hoạt động, độ phức tạp thời gian và không gian, cũng như mức độ tối ưu của từng phương pháp. Ngoài ra, các phương pháp được đánh giá định lượng trên các cấu hình Phase 1 thông qua các chỉ số như net reward, phần trăm đơn giao đúng hạn và thời gian chạy. Từ đó, báo cáo đưa ra phân tích trade-off giữa chất lượng nghiệm, khả năng mở rộng, độ ổn định và chi phí tính toán. Bên cạnh đó, báo cáo cũng trình bày chiến lược xử lý các tình huống surge và hotspot, tức các thời điểm có nhiều đơn hàng xuất hiện hoặc tập trung tại một khu vực nhất định, nhằm cải thiện khả năng thích nghi của solver trong môi trường thực tế.

II. MÔ HÌNH HÓA BÀI TOÁN

Bài toán được mô hình hóa trên một bản đồ dạng lưới A kích thước $N \times N$. Mỗi ô trên bản đồ được ký hiệu là $A[i][j]$, trong đó $A[i][j] = 0$ là ô trống và $A[i][j] = 1$ là ô vật cản. Shipper chỉ được di chuyển trên các ô trống và không được đi ra ngoài phạm vi bản đồ.

Tại thời điểm ban đầu $t = 0$, có C shipper trên bản đồ. Shipper thứ i có vị trí ban đầu (x_i, y_i) , tải trọng tối đa $W_{\max}(i)$

và sức chứa tối đa $K(i)$ đơn hàng. Tại mọi thời điểm, không có hai shipper nào được đứng trên cùng một ô.

A. Tập hành động

Tại mỗi bước thời gian t , mỗi shipper thực hiện một cặp hành động:

$$a_i(t) = (move_i(t), cargo_op_i(t)).$$

Trong đó:

$$move_i(t) \in \{S, L, R, U, D\},$$

với S là đứng yên, L là sang trái, R là sang phải, U là đi lên và D là đi xuống một ô.

Thành phần $cargo_op_i(t)$ biểu diễn thao tác với hàng hóa:

$$cargo_op_i(t) \in \{0, 1, 2\},$$

trong đó 0 là không thao tác, 1 là nhặt đơn tại ô hiện tại, và 2 là giao một đơn đang mang nếu shipper đang đứng tại đúng vị trí giao hàng của đơn đó.

Thứ tự thực hiện trong một bước là:

Di chuyển $\rightarrow$ Thao tác hàng hóa.

Nói cách khác, shipper trước hết di chuyển theo $move_i(t)$, sau đó thực hiện thao tác nhặt hoặc giao hàng tại ô sau khi di chuyển.

B. Mô hình đơn hàng

Tập đơn hàng gồm G đơn:

$$\mathcal{G} = \{g_1, g_2, \dots, g_G\}.$$

Mỗi đơn hàng g_i được biểu diễn bởi:

$$g_i = \langle sx_i, sy_i, ex_i, ey_i, et_i, w_i, p_i \rangle.$$

Trong đó, (sx_i, sy_i) là tọa độ điểm lấy hàng, (ex_i, ey_i) là tọa độ điểm giao hàng, et_i là deadline, w_i là khối lượng kiện hàng và $p_i \in \{1, 2, 3\}$ là mức ưu tiên của đơn hàng. Cụ thể, $p_i = 1$ là đơn tiêu chuẩn, $p_i = 2$ là đơn nhanh và $p_i = 3$ là đơn hỏa tốc.

Tại mỗi thời điểm, một đơn hàng có thể thuộc một trong ba trạng thái: chưa được nhặt, đang được một shipper mang, hoặc đã được giao.

C. Mô hình sinh đơn hàng: Surge và Hotspot

Đơn hàng xuất hiện theo quá trình Poisson không đồng nhất với tốc độ sinh đơn $\lambda(t)$ thay đổi theo thời gian. Gọi λ_0 là tốc độ sinh đơn nền, xấp xỉ:

$$\lambda_0 \approx \frac{G}{T}.$$

Nếu t nằm trong surge window $[t_s, t_e]$, tốc độ sinh đơn được khuếch đại bởi hệ số surge A :

$$\lambda(t) = \begin{cases} \lambda_0(1 + A), & \text{nếu } t \in [t_s, t_e], \\ \lambda_0, & \text{ngược lại.} \end{cases}$$

Ngoài surge theo thời gian, bài toán còn có hotspot theo không gian. Gọi (r, c) là tâm hotspot. Trong surge window,

với xác suất 70%, điểm lấy hàng (sx_i, sy_i) được chọn trong vùng lân cận Manhattan không quá 3 quanh hotspot:

$$|sx_i - r| + |sy_i - c| \leq 3.$$

Với xác suất 30% còn lại, điểm lấy hàng được chọn ngẫu nhiên trên toàn bản đồ. Cơ chế này tạo ra các vùng có mật độ đơn cao, khiến nhiều shipper có thể cùng cạnh tranh tài nguyên di chuyển và pickup tại một khu vực hẹp.

Trong Phase 1, các tham số như λ_0 , A , danh sách surge window và hotspot không được công bố. Điều này yêu cầu thuật toán phải có khả năng thích nghi với môi trường động thay vì hard-code theo cấu hình biết trước. Trong Phase 2, các tham số surge và hotspot được công bố đầy đủ.

D. Ràng buộc sức chứa và trọng lượng

Tại mọi thời điểm t , mỗi shipper i phải thỏa mãn đồng thời hai ràng buộc. Thứ nhất, tổng khối lượng các đơn trong túi không được vượt quá tải trọng tối đa:

$$\sum_{g_j \in bag(i, t)} w_j \leq W_{\max}(i).$$

Thứ hai, số lượng đơn trong túi không được vượt quá sức chứa:

$$|bag(i, t)| \leq K(i).$$

Khi shipper nhặt hàng tại một ô có nhiều đơn, thứ tự ưu tiên được xác định theo mức ưu tiên của đơn hàng: hỏa tốc trước, sau đó đến nhanh, sau đó đến tiêu chuẩn. Nếu nhiều đơn có cùng mức ưu tiên, đơn có chỉ số nhỏ hơn được ưu tiên.

E. Hàm phần thưởng

Giả sử đơn hàng g_i được giao tại thời điểm $t_{\text{delivery}}(i)$. Phần thưởng giao đơn $r(i)$ phụ thuộc vào việc đơn được giao đúng hạn hay trễ hạn.

Nếu đơn được giao đúng hạn, tức:

$$t_{\text{delivery}}(i) \leq et_i,$$

thì:

$$r(i) = \alpha_{p_i} \cdot r_{\text{base}}(i) \cdot (1 + \text{bonus}_i),$$

trong đó:

$$\text{bonus}_i = \max\left(0, \frac{et_i - t_{\text{delivery}}(i)}{et_i}\right).$$

Nếu đơn bị giao trễ, tức:

$$t_{\text{delivery}}(i) > et_i,$$

thì:

$$r(i) = \beta_{p_i} \cdot r_{\text{base}}(i) \cdot \max\left(0, 1 - \frac{t_{\text{delivery}}(i) - et_i}{T}\right).$$

Các hệ số α_{p_i} và β_{p_i} phụ thuộc vào mức ưu tiên p_i . Với đơn hỏa tốc $p_i = 3$, $\alpha_{p_i} = 3.0$ và $\beta_{p_i} = 0.5$. Với đơn nhanh $p_i = 2$, $\alpha_{p_i} = 2.0$ và $\beta_{p_i} = 0.3$. Với đơn tiêu chuẩn $p_i = 1$, $\alpha_{p_i} = 1.0$ và $\beta_{p_i} = 0.1$.

Phần thưởng cơ bản của đơn hàng được tính theo:

$$r_{\text{base}}(i) = 10 \times f_{\text{weight}}(w_i),$$

trong đó $f_{\text{weight}}(w_i)$ phụ thuộc vào khối lượng w_i của đơn hàng.

F. Chi phí di chuyển

Chi phí di chuyển của shipper i tại bước t được ký hiệu là $rc(i, t)$. Nếu shipper đứng yên, chi phí bằng 0. Nếu shipper di chuyển bằng một trong các hành động L , R , U hoặc D , chi phí được tính như sau:

$$rc(i, t) = -0.01 \left(1 + \gamma \frac{W_{\text{carried}}(i, t)}{W_{\text{max}}(i)} \right),$$

với $\gamma = 1.0$.

Trong đó, $W_{\text{carried}}(i, t)$ là tổng khối lượng các đơn mà shipper i đang mang tại thời điểm t :

$$W_{\text{carried}}(i, t) = \sum_{g_j \in \text{bag}(i, t)} w_j.$$

Do $rc(i, t) \leq 0$, đây là chi phí âm được cộng vào tổng điểm thưởng ròng.

G. Hàm mục tiêu

Mục tiêu của bài toán là tìm chuỗi hành động cho toàn bộ shipper trong khoảng thời gian mô phỏng T sao cho tổng net reward là lớn nhất:

$$\max \left[\sum_{g_i \in \mathcal{G}_{\text{delivered}}} r(i) + \sum_{t=0}^{T-1} \sum_{i=1}^C rc(i, t) \right].$$

Trong đó, $\mathcal{G}_{\text{delivered}}$ là tập các đơn đã được giao thành công trong toàn bộ quá trình mô phỏng. Vì $rc(i, t)$ là chi phí âm khi di chuyển, hàm mục tiêu trên tương đương với việc tối đa hóa tổng phần thưởng giao hàng sau khi trừ đi chi phí di chuyển.

H. Các ràng buộc vận hành

Trong quá trình mô phỏng, các ràng buộc vận hành sau phải luôn được thỏa mãn:

- Shipper không được đi ra ngoài bản đồ.
- Shipper không được đi vào ô vật cản, tức các ô có $A[i][j] = 1$.
- Không có hai shipper nào được đứng trên cùng một ô tại cùng một thời điểm.
- Khi xảy ra tranh chấp vị trí, shipper có chỉ số nhỏ hơn được ưu tiên theo luật của môi trường.
- Mỗi shipper phải luôn thỏa mãn đồng thời ràng buộc tải trọng $W_{\text{max}}(i)$ và sức chứa $K(i)$.
- Thao tác nhặt hàng chỉ hợp lệ nếu shipper đứng tại điểm lấy hàng của một đơn chưa được nhặt.
- Thao tác giao hàng chỉ hợp lệ nếu shipper đứng tại điểm giao hàng của một đơn đang nằm trong túi của shipper đó.

Do đơn hàng có thể được tiết lộ dần theo thời gian, bài toán mang tính online. Tại thời điểm t , thuật toán chỉ được sử dụng thông tin hiện có trong quan sát hiện tại để đưa ra hành động cho timestep đó. Vì vậy, thay vì giải tối ưu toàn cục với toàn bộ thông tin tương lai, các phương pháp trong báo cáo tập trung vào lập kế hoạch xấp xỉ, tái lập kế hoạch chọn lọc và cân bằng giữa chất lượng nghiệm với thời gian tính toán.

III. PHƯƠNG PHÁP ĐỀ XUẤT

A. Greedy BFS

1) *Ý tưởng chính*: Phương pháp GreedyBFS xem bài toán giao hàng nhiều shipper như một bài toán gán nhiệm vụ động trên bản đồ lưới. Ở mỗi timestep, solver chọn hành động cho từng shipper dựa trên trạng thái hiện tại của đơn hàng, vị trí shipper và các đơn đang được mang. Vì đơn hàng xuất hiện theo thời gian, solver không lập kế hoạch tối ưu dài hạn mà sử dụng chiến lược greedy để chọn nhiệm vụ phù hợp tại từng thời điểm. Solver sử dụng BFS để tìm đường đi ngắn nhất giữa hai vị trí trên bản đồ. Các kết quả khoảng cách và bước đi đầu tiên được lưu trong `dist_cache` và `move_cache`, giúp giảm chi phí khi phải truy vấn lại các cặp vị trí đã xét trước đó. Tại mỗi timestep, solver ưu tiên xử lý các hành động tức thời. Nếu shipper đang ở điểm giao của một đơn trong túi hàng, shipper sẽ thực hiện delivery. Nếu shipper đang ở điểm pickup của một đơn hợp lệ, solver sẽ pickup đơn có độ ưu tiên cao hơn, deadline sớm hơn và id nhỏ hơn. Với các shipper chưa có hành động, solver xây dựng danh sách target gồm các điểm delivery của đơn đang mang và các điểm pickup của đơn mới có thể nhận. Mỗi target được đánh giá dựa trên reward ước lượng, độ ưu tiên, khoảng cách di chuyển và lợi ích dự kiến của nhiệm vụ kế tiếp. Reward dự kiến được mô tả như sau:

$$R(o, t) = \begin{cases} \alpha \cdot r_b \cdot \left(1 + \frac{e_t - t}{\max(e_t, 1)} \right), & t \leq e_t \\ \beta \cdot r_b \cdot \max \left(0, 1 - \frac{t - e_t}{T} \right), & t > e_t \end{cases}$$

Điểm đánh giá nhiệm vụ có thể viết dưới dạng:

$$\text{score} = \frac{R_1 + 0.6R_2 + p \cdot B}{d + \varepsilon}$$

Trong đó R_1 là reward của nhiệm vụ hiện tại, R_2 là reward ước lượng cho nhiệm vụ kế tiếp, p là độ ưu tiên, B là hệ số thưởng ưu tiên, d là khoảng cách tới target và ε là hằng số nhỏ để tránh chia cho 0. Solver chọn cặp shipper–nhiệm vụ có điểm cao nhất, gán nhiệm vụ đó rồi lặp lại cho các shipper còn lại. Nếu không tìm được nhiệm vụ phù hợp, solver dùng fallback bằng cách chọn một đơn hợp lệ còn lại để di chuyển tới pickup. Cuối cùng, solver xử lý tránh va chạm cục bộ. Các shipper được xét theo độ gấp deadline, số đơn đang mang và id. Nếu ô tiếp theo đã bị giữ chỗ, solver tìm hướng thay thế gần mục tiêu hơn và hạn chế đi qua các ô bottleneck; nếu không có hướng phù hợp thì shipper đứng yên.

2) *Độ phức tạp*: Gọi N là kích thước bản đồ, $V = N^2$ là số ô trên lưới, C là số shipper, G là số đơn hàng đang active, B là sức chứa tối đa của shipper, K_1 là số target chính được xét ở bước đầu và K_2 là số target kế tiếp được xét để ước lượng lợi ích tương lai. Mỗi lần BFS giữa hai vị trí có chi phí:

$$O(V) = O(N^2)$$

Sau khi kết quả đã được lưu trong cache, truy vấn khoảng cách hoặc hướng đi đầu tiên có chi phí xấp xỉ:

$$O(1)$$

Ở mỗi timestep, solver xử lý hành động tức thời cho toàn bộ shipper. Với mỗi shipper, solver kiểm tra các đơn đang mang và các đơn có thể pickup tại vị trí hiện tại, nên chi phí xấp xỉ:

$$O(C(B + G))$$

Trong bước gán nhiệm vụ lặp toàn cục, mỗi shipper chưa được gán cần xây danh sách target từ các đơn đang mang và các đơn active có thể nhận. Các target được sắp xếp theo khoảng cách, sau đó solver xét một số target đầu tiên và đánh giá thêm một bước nhiệm vụ kế tiếp. Chi phí đánh giá một shipper có thể viết gần đúng là:

$$O(G \log G + K_1 K_2)$$

Vì quá trình gán nhiệm vụ chọn từng shipper một theo cặp có điểm cao nhất, trong trường hợp có tối đa C shipper cần gán, chi phí gần đúng của bước global iterative assignment là:

$$O(C^2(G \log G + K_1 K_2))$$

Bước fallback cho các shipper còn lại có thể phải duyệt qua các đơn active, nên có chi phí:

$$O(CG)$$

Bước tránh va chạm cần sắp xếp shipper theo độ gấp deadline và kiểm tra vị trí đã được giữ chỗ. Việc tính slack có thể xét các đơn trong túi hàng, do đó chi phí xấp xỉ:

$$O(CB + C \log C)$$

Vì vậy, độ phức tạp gần đúng cho một timestep là:

$$O(C(B + G) + C^2(G \log G + K_1 K_2) + CG + CB + C \log C)$$

Có thể rút gọn thành:

$$O(C^2(G \log G + K_1 K_2))$$

chưa tính các lần BFS cache miss. Nếu có S cặp vị trí mới cần chạy BFS trong timestep đó, cần cộng thêm:

$$O(SN^2)$$

Về không gian, solver lưu cache khoảng cách và bước đi đầu tiên cho các cặp vị trí đã truy vấn. Nếu có S cặp vị trí được lưu, bộ nhớ cho cache là:

$$O(S)$$

Ngoài ra, mỗi lần BFS cần lưu hàng đợi và mảng parent trên tối đa N^2 ô, nên chi phí phụ trong một lần BFS là:

$$O(N^2)$$

Do đó, độ phức tạp không gian tổng quát là:

$$O(S + N^2)$$

3) *Mức độ tối ưu*: GreedyBFS là một phương pháp heuristic trực tuyến, được thiết kế để đưa ra quyết định nhanh ở từng timestep thay vì tìm nghiệm tối ưu toàn cục. Do đơn hàng có thể xuất hiện theo thời gian, vị trí shipper thay đổi liên tục và trạng thái túi hàng không cố định, solver chỉ dựa trên thông tin hiện có để chọn hành động phù hợp tại thời điểm xét.

Một điểm mạnh của solver là sử dụng BFS để tính đường đi ngắn nhất giữa hai vị trí trên bản đồ lưới. Nếu bản đồ không thay đổi và mọi bước di chuyển có cùng chi phí, BFS đảm bảo tìm được đường đi ngắn nhất cho từng cặp vị trí. Nhờ đó, các giá trị khoảng cách dùng trong quá trình chọn nhiệm vụ và xác định bước di chuyển tiếp theo có độ chính xác cao trong phạm vi bản đồ hiện tại.

Tuy nhiên, phần chọn nhiệm vụ không phải là một phép tối ưu chính xác. Solver đánh giá nhiệm vụ bằng hàm điểm heuristic, kết hợp reward ước lượng, độ ưu tiên của đơn, khoảng cách tới target và lợi ích dự kiến của nhiệm vụ kế tiếp. Cơ chế nhìn trước ngắn hạn này giúp solver tránh việc chỉ chọn đơn gần nhất, nhưng vẫn có thể bỏ qua những phương án tốt hơn trong dài hạn do không xét toàn bộ chuỗi pickup-delivery có thể có.

Cơ chế tránh va chạm cũng được xử lý theo hướng cục bộ. Solver ưu tiên các shipper có deadline gấp hơn hoặc đang mang nhiều đơn hơn, sau đó tìm hướng thay thế nếu ô tiếp theo đã bị giữ chỗ. Cách làm này giúp giảm xung đột và giữ tốc độ xử lý nhanh, nhưng không đảm bảo tạo ra kế hoạch di chuyển tối ưu cho toàn bộ đội shipper.

Nhìn chung, GreedyBFS có thể được xem là một **online greedy heuristic solver with BFS-based shortest path**. Thuật toán có ưu điểm là đơn giản, tốc độ nhanh, dễ triển khai và phù hợp với môi trường online. Đổi lại, chất lượng nghiệm phụ thuộc vào hàm scoring, giới hạn số target được xét, mức độ chính xác của reward ước lượng và chiến lược xử lý va chạm tại từng timestep.

B. VRP-OrTools

1) *Ý tưởng chính*: Phương pháp VRP-OrTools xem bài toán giao hàng nhiều shipper như một biến thể của Vehicle Routing Problem với pickup-delivery, ràng buộc tải trọng, deadline và chi phí di chuyển. Tuy nhiên, do bài toán có tính online, đơn hàng có thể xuất hiện theo thời gian, nên việc giải lại toàn bộ VRP ở mỗi timestep là không hiệu quả. Vì vậy, solver được thiết kế theo hướng heuristic động: giữ lại kế hoạch cũ nếu còn tốt, chỉ tái lập kế hoạch khi có sự kiện quan trọng, và chỉ rebuild các shipper bị ảnh hưởng.

Solver sử dụng cache đường đi ngắn nhất bằng BFS một nguồn. Với mỗi vị trí nguồn, BFS được chạy một lần để tính khoảng cách tới toàn bộ ô hợp lệ trên bản đồ. Sau đó, các truy vấn khoảng cách được lấy trực tiếp từ cache, giúp giảm chi phí tính toán khi scoring order, chọn candidate và xây dựng route.

Tại mỗi lần replan, solver trước hết lọc các đơn hàng tiềm năng bằng candidate pruning. Các pickup candidate được chấm điểm nhanh dựa trên độ ưu tiên, khoảng cách tới pickup, độ gấp của deadline và cluster bonus nếu đơn nằm trong vùng có nhiều đơn gần nhau. Sau đó, solver xây dựng ma trận lợi nhuận

giữa các shipper cần rebuild và các target, bao gồm cả pickup, delivery và dummy wait columns. Hungarian algorithm được dùng để gán mỗi shipper với một target chính sao cho tổng lợi nhuận là lớn nhất trên tập candidate đã lọc.

Sau khi có target chính, solver xây route bằng sequential multi-stop route builder. Route builder mô phỏng trạng thái ảo của shipper gồm vị trí hiện tại, thời gian ước lượng và túi hàng hiện tại. Ở mỗi bước, nó chọn giữa việc giao đơn đang mang hoặc lấy thêm đơn mới, dựa trên reward dự kiến, deadline, chi phí di chuyển, tải trọng, cluster bonus và delay cost lên các đơn đang mang. Nhờ đó, route có thể xen kẽ nhiều hành động như:

pickup → pickup → delivery → pickup → delivery.

Độ sâu route $L_{\max}$ được tính động theo kích thước bản đồ, sức chứa trung bình của shipper và loại replan. Gọi $\bar{K}$ là sức chứa trung bình của đội shipper:

$$\bar{K} = \frac{1}{C} \sum_{i=1}^C K(i).$$

Trước hết, solver tính độ sâu cơ sở:

$$L_{\max}^{base} = 4 + \left\lfloor \frac{N}{25} \right\rfloor + \left\lfloor \frac{\bar{K}}{2} \right\rfloor.$$

Nếu lần tái lập kế hoạch hiện tại là global replan, solver tăng độ sâu route để phối hợp toàn đội:

$$L_{\max}^{base} \leftarrow L_{\max}^{base} + 2.$$

Nếu là local replan, solver giảm độ sâu route để phản ứng nhanh hơn:

$$L_{\max}^{base} \leftarrow L_{\max}^{base} - 1.$$

Sau đó, runtime throttle được áp dụng thông qua hệ số phạt λ , phản ánh mức độ chậm của các lần replan gần đây:

$$L_{\max} = \text{clip} \left(L_{\max}^{base} - \left\lfloor \frac{\lambda}{2} \right\rfloor, 3, 12 \right).$$

Trong đó $\text{clip}(x, a, b)$ giới hạn giá trị x trong đoạn $[a, b]$. Khi thời gian replan gần đây quá lớn, λ tăng lên, làm giảm $L_{\max}$, số candidate K và số candidate kế tiếp K_{next} để tránh vượt giới hạn thời gian. Ngược lại, khi solver chạy nhanh, λ giảm dần để cho phép route builder tìm kiếm sâu hơn.

Cuối cùng, solver dùng predictive score guard để so sánh kế hoạch mới, kế hoạch cũ và kế hoạch fallback. Kế hoạch có điểm dự đoán tốt nhất sẽ được chọn. Sau đó, một conflict resolver heuristic xử lý các xung đột di chuyển giữa shipper bằng cách ưu tiên shipper đang mang nhiều đơn hoặc có deadline gấp.

2) *Độ phức tạp*: Gọi N là kích thước bản đồ, $V = N^2$ là số ô trên lưới, C là số shipper, G là số đơn hàng đang active, R là số shipper cần rebuild tại một lần replan, K là số pickup candidate giữ lại cho mỗi shipper, M là số target sau pruning, L là độ sâu route và K_{next} là số pickup candidate xét tại mỗi bước route.

Với BFS cache, một lần cache miss từ một nguồn có chi phí:

$$O(N^2).$$

Sau khi đã cache, truy vấn khoảng cách có chi phí xấp xỉ:

$$O(1).$$

Candidate pruning cho R shipper có chi phí:

$$O(RG \log G),$$

do mỗi shipper cần chấm điểm và sắp xếp các đơn active để lấy top- K .

Hungarian assignment trên ma trận $R \times (M + R)$ có độ phức tạp xấp xỉ:

$$O(R^2(M + R)).$$

Sequential route builder hiện tại có thể phải xét lại các đơn active ở mỗi bước route. Do đó, chi phí xấp xỉ:

$$O(RLG \log G).$$

Nếu route builder chỉ dùng tập candidate đã prune, chi phí có thể giảm còn:

$$O(RLK_{\text{next}}).$$

Conflict resolver có chi phí khoảng:

$$O(C^2).$$

Vì vậy, độ phức tạp mỗi lần replan có thể viết gần đúng là:

$$O(RG \log G + R^2(M + R) + RLG \log G + C^2),$$

chưa tính các lần BFS cache miss. Trong các timestep không cần replan, solver chỉ đi theo kế hoạch cũ và xử lý conflict, nên chi phí giảm còn khoảng:

$$O(C + C^2).$$

Về không gian, solver cần:

$$O(SN^2)$$

cho cache BFS với S nguồn đã lưu,

$$O(R(M + R))$$

cho ma trận Hungarian, và

$$O(CL)$$

cho kế hoạch hiện tại của các shipper.

3) *Mức độ tối ưu*: Thuật toán VRP-OrTools trong phiên bản hiện tại thuộc nhóm *heuristic*. Solver không đảm bảo tối ưu toàn cục cho toàn bộ bài toán online pickup-delivery vì đơn hàng được tiết lộ theo thời gian, tập candidate bị cắt giảm bởi candidate pruning, route builder lựa chọn tuần tự theo điểm heuristic, và conflict resolver chỉ xử lý va chạm theo luật ưu tiên cục bộ.

Tuy nhiên, một số thành phần con có đảm bảo tối ưu trong phạm vi hẹp. Thứ nhất, BFS tìm được đường đi ngắn nhất giữa hai ô nếu bản đồ là lưới tĩnh và mọi bước di chuyển có chi phí bằng nhau. Thứ hai, Hungarian/Kuhn-Munkres tìm assignment tối ưu trên ma trận lợi nhuận đã được xây dựng. Tuy nhiên, do ma trận này chỉ chứa các target sau bước pruning, kết quả chỉ tối ưu trên tập candidate đã chọn, không tối ưu trên toàn bộ tập đơn hàng. Thứ ba, hàm tối ưu thứ tự giao hàng dùng hoán vị đầy đủ khi số đơn trong túi nhỏ hơn hoặc bằng ngưỡng `perm_limit`; trong trường hợp này, thứ tự giao hàng là tối ưu theo hàm điểm nội bộ. Khi số đơn vượt quá ngưỡng này, solver chuyển sang greedy ordering nên không còn đảm bảo tối ưu.

Vì vậy, có thể phân loại phương pháp này là một *online heuristic solver with locally optimal subroutines*. Nó không đảm bảo optimal toàn cục, nhưng đạt near-optimal cục bộ trong các bước như shortest-path search, assignment trên tập candidate, và delivery ordering với số lượng đơn nhỏ.

C. Multi-Agent Pickup and Delivery với Conflict-Based Search

1) *Ý tưởng chính*: Solver MAPD-CBS xem bài toán giao hàng nhiều shipper như một bài toán *Multi-Agent Pickup and Delivery* trên bản đồ lưới, với các ràng buộc về tải trọng, số lượng đơn, deadline và tránh va chạm. Vì đơn hàng xuất hiện theo thời gian, solver không giải lại toàn bộ bài toán ở mỗi timestep mà sử dụng heuristic động: giữ nhiệm vụ cũ nếu còn hợp lệ, chỉ gán lại khi cần và ưu tiên shipper đang mang hàng hoặc các đơn sắp trễ.

Solver sử dụng **BFS cache** để lưu khoảng cách ngắn nhất giữa các vị trí trên bản đồ. Ban đầu, khoảng cách nhanh có thể được ước lượng bằng công thức Manhattan:

$$d(a, b) = |x_a - x_b| + |y_a - y_b|$$

Sau đó, các candidate tốt sẽ được đánh giá lại bằng khoảng cách BFS chính xác. Với mỗi đơn hàng, thời gian hoàn thành dự kiến được tính gần đúng như sau:

$$finish = t + d(shipper, pickup) + d(pickup, delivery) + 1$$

Trong đó t là thời điểm hiện tại, $d(shipper, pickup)$ là khoảng cách từ shipper đến điểm lấy hàng, $d(pickup, delivery)$ là khoảng cách từ điểm lấy đến điểm giao, và 1 là thời gian thực hiện thao tác pickup.

Để xét mức độ gấp của đơn hàng, solver dùng độ trễ và slack deadline:

$$late = \max(0, finish - deadline)$$

$$slack = deadline - finish$$

Nếu $late = 0$, đơn được giao đúng hạn; nếu $late > 0$, đơn bị trễ. Ngược lại, $slack$ càng nhỏ thì đơn càng gấp và cần được ưu tiên hơn.

Với shipper rảnh, solver lọc và chấm điểm các đơn hàng tiềm năng dựa trên reward, độ ưu tiên, khoảng cách, deadline và mật độ đơn gần nhau. Hàm điểm có thể biểu diễn tổng quát như sau:

$$score = R + \alpha p + \beta C - \gamma d - \lambda late$$

Trong đó R là reward dự kiến, p là độ ưu tiên của đơn, C là cluster bonus khi đơn nằm gần nhiều đơn khác, d là tổng quãng đường di chuyển, $late$ là độ trễ, còn $\alpha, \beta, \gamma, \lambda$ là các hệ số điều chỉnh. Đơn có $score$ cao hơn sẽ được ưu tiên gán cho shipper.

Với shipper đang mang hàng, solver chọn đơn cần giao trước dựa trên reward dự kiến, khoảng cách tối điểm giao và độ gấp của deadline. Nếu có đơn mới nằm tiện đường, solver có thể cho shipper lấy thêm, miễn là không vi phạm ràng buộc tải trọng và số lượng đơn:

$$current_weight + order_weight \leq W_{max}$$

$$current_orders + 1 \leq K_{max}$$

Sau khi chọn mục tiêu, solver lập đường đi bằng cơ chế **reservation theo thời gian** để hạn chế nhiều shipper đi vào cùng một ô hoặc đổi chỗ trực tiếp. Nếu vẫn xảy ra xung đột, conflict resolver sẽ ưu tiên shipper có deadline gấp hơn hoặc nhiệm vụ quan trọng hơn, còn shipper khác sẽ đổi hướng hoặc đứng yên. Nhờ kết hợp giữa gán đơn động, BFS cache, hàm scoring heuristic và xử lý xung đột cục bộ, solver có thể phản ứng nhanh với đơn hàng online, đồng thời giảm va chạm và cải thiện hiệu quả giao hàng.

2) *Độ phức tạp*: Gọi N là kích thước bản đồ, $V = N^2$ là số ô trên lưới, C là số shipper, G là số đơn hàng đang active, P là số shipper rảnh cần gán đơn, K là số candidate được giữ lại để đánh giá chính xác, và H là độ dài horizon khi lập đường đi.

Solver sử dụng cache khoảng cách ngắn nhất bằng BFS. Mỗi lần cache miss, chi phí chạy BFS là:

$$O(V) = O(N^2)$$

Sau khi đã cache, mỗi truy vấn khoảng cách có chi phí xấp xỉ:

$$O(1)$$

Ở bước gán đơn, mỗi shipper rảnh duyệt qua tối đa G đơn hàng và sắp xếp để lấy các candidate tốt nhất, nên chi phí candidate pruning là:

$$O(PG \log G)$$

Sau đó, solver chỉ đánh giá chính xác K candidate cho mỗi shipper, nên chi phí là:

$$O(PK)$$

Việc phối hợp gán đơn được thực hiện bằng cách sắp xếp danh sách candidate theo điểm số và regret. Với khoảng PK candidate, chi phí là:

$$O(PK \log(PK))$$

Với các shipper đang mang hàng, solver chọn đơn cần giao trước bằng cách xét các đơn trong túi hàng. Nếu B là sức chứa tối đa của shipper, chi phí xấp xỉ:

$$O(CB)$$

Sau khi có mục tiêu, solver lập đường đi bằng reservation trong horizon H . Do mỗi bước chỉ xét số hướng di chuyển cố định, chi phí cơ bản là:

$$O(CH)$$

Conflict resolver kiểm tra xung đột giữa các shipper, gồm xung đột cùng ô và đổi chỗ trực tiếp, nên chi phí là:

$$O(C^2)$$

Vì vậy, độ phức tạp gần đúng cho một timestep là:

$$O(PG \log G + PK + PK \log(PK) + CB + CH + C^2)$$

chưa tính các lần BFS cache miss. Nếu có S đích mới cần chạy BFS, cần cộng thêm:

$$O(SN^2)$$

Trong trường hợp nhiều shipper rảnh và nhiều đơn active, thành phần chi phối thường là:

$$O(PG \log G + C^2)$$

Về không gian, solver cần lưu cache BFS cho tối đa S trường khoảng cách, mỗi trường có tối đa N^2 ô, cùng với kế hoạch tạm thời trong horizon H . Do đó, độ phức tạp không gian là:

$$O(SN^2 + CH)$$

3) *Mức độ tối ưu*: Phương pháp MAPD-CBS được xây dựng theo hướng **heuristic trực tuyến**, phù hợp với đặc điểm của bài toán giao hàng có nhiều shipper và đơn hàng xuất hiện động theo thời gian. Do trạng thái hệ thống luôn thay đổi sau mỗi timestep, solver không hướng tới việc tìm nghiệm tối ưu toàn cục cho toàn bộ quá trình. Thay vào đó, thuật toán tập trung đưa ra quyết định đủ tốt trong thời gian ngắn, dựa trên thông tin hiện có tại thời điểm xét.

Một số thành phần trong solver vẫn có tính tối ưu cục bộ. Cụ thể, BFS cache cho phép tính khoảng cách ngắn nhất giữa các ô trên bản đồ lưới tĩnh, với giả thiết mỗi bước di chuyển

có cùng chi phí. Nhờ đó, các khoảng cách dùng để đánh giá đơn hàng, chọn điểm pickup, delivery và xây dựng hướng di chuyển được đảm bảo chính xác trong phạm vi bản đồ hiện tại.

Tuy nhiên, quá trình gán đơn và chọn mục tiêu chủ yếu dựa trên các hàm điểm heuristic. Solver đánh giá đơn hàng thông qua nhiều yếu tố như reward dự kiến, độ ưu tiên, khoảng cách di chuyển, deadline, độ trễ và mật độ các đơn gần nhau. Vì chỉ xét các candidate đã được lọc thay vì toàn bộ không gian nghiệm, thuật toán có thể bỏ qua một số phương án tốt hơn về dài hạn.

Đối với việc di chuyển của nhiều shipper, solver sử dụng cơ chế reservation theo thời gian kết hợp CBS-lite để giảm xung đột. Cách tiếp cận này không đảm bảo tìm được kế hoạch di chuyển tối ưu cho toàn bộ đội shipper, nhưng có hiệu quả trong việc hạn chế các va chạm thường gặp như nhiều shipper cùng đi vào một ô hoặc hai shipper đổi chỗ trực tiếp cho nhau.

Nhìn chung, phương pháp này có thể xem là một **online MAPD heuristic solver with CBS-lite conflict handling**. Ưu điểm chính của solver là tốc độ phản ứng nhanh, khả năng xử lý nhiều shipper trong môi trường online và chi phí tính toán hợp lý. Đối lại, chất lượng nghiệm phụ thuộc vào các tham số như số candidate được xét, độ dài horizon, trọng số trong hàm scoring, cơ chế ưu tiên deadline và cách xử lý xung đột.

D. Ant Colony Optimization

1) *Ý tưởng chính*: Ant Colony Optimization, viết tắt là ACO, là một metaheuristic mô phỏng cách đàn kiến tìm đường thông qua pheromone. Trong bài toán này, ACO không được dùng để giải toàn bộ bài toán định tuyến cho cả fleet từ đầu, vì cách đó có chi phí rất lớn trong môi trường online. Thay vào đó, ACO được dùng như một local route improver cho từng shipper cần tái lập kế hoạch.

Pipeline tổng quát của solver ACO vẫn giữ các thành phần online quan trọng như plan preservation, selective replanning, candidate pruning, Hungarian assignment và conflict resolver. Cụ thể, trước tiên solver kiểm tra kế hoạch hiện tại của các shipper. Nếu không có sự kiện quan trọng, shipper tiếp tục đi theo kế hoạch cũ. Nếu cần replan, solver chỉ chọn các shipper bị ảnh hưởng, sau đó dùng Hungarian assignment để gán mỗi shipper với một target chính. Sau bước này, ACO được dùng để xây dựng hoặc cải thiện route nhiều điểm cho từng shipper.

Với mỗi shipper, ACO bắt đầu từ trạng thái hiện tại gồm vị trí, thời gian và túi hàng đang mang. Nếu shipper đã được Hungarian assignment gán một primary target, target này được đưa vào route trước. Sau đó, mỗi ant xây dựng một route bằng cách lặp lại quá trình chọn target tiếp theo. Ở mỗi bước, tập candidate gồm:

- các delivery candidate từ các đơn đang nằm trong túi hàng ảo;
- các pickup candidate khả thi từ tập đơn đã được pruning.

Xác suất chọn target tiếp theo được tính theo công thức ACO:

$$P(i \rightarrow j) \propto \tau_{ij}^\alpha \eta_{ij}^\beta,$$

trong đó τ_{ij} là pheromone trên cạnh chuyển tiếp từ target i sang target j , còn η_{ij} là heuristic desirability của target j .

Trong implementation hiện tại, pheromone được lưu cục bộ trong mỗi lần gọi route builder, giúp tránh việc pheromone cũ bị lỗi thời khi đơn hàng biến mất hoặc được shipper khác xử lý.

Heuristic desirability η_{ij} không chỉ phụ thuộc vào khoảng cách, mà còn được xây dựng từ reward dự kiến, deadline, độ ưu tiên, chi phí di chuyển, cluster bonus và delay cost lên các đơn đang mang. Do đó, ACO có thể khám phá nhiều chuỗi route khác nhau, chẳng hạn:

pickup $\rightarrow$ pickup $\rightarrow$ delivery $\rightarrow$ pickup $\rightarrow$ delivery.

Để tránh trường hợp ACO sinh route kém hơn heuristic tham lam, solver luôn xây dựng một greedy route bằng sequential route builder làm baseline. ACO chỉ được chọn nếu route tốt nhất từ các ant có điểm dự đoán cao hơn greedy baseline. Cơ chế này giúp ACO đóng vai trò cải thiện route thay vì thay thế hoàn toàn chiến lược heuristic ổn định.

2) *Độ phức tạp*: Gọi N là kích thước bản đồ, $V = N^2$ là số ô trên lưới, C là số shipper, G là số đơn active, R là số shipper cần rebuild trong một lần replan, K là số pickup candidate sau pruning, L là độ sâu route tối đa, K_{next} là số pickup candidate xét tại mỗi bước route, A là số ant và I là số vòng lặp ACO.

Với BFS cache, một lần cache miss có chi phí:

$$O(N^2).$$

Sau khi đã cache, truy vấn khoảng cách có chi phí xấp xỉ:

$$O(1).$$

Candidate pruning cho R shipper có chi phí:

$$O(RG \log G),$$

do mỗi shipper cần chấm điểm và sắp xếp các đơn active để lấy top- K .

Trước khi chạy ACO, solver vẫn dùng Hungarian assignment để chọn primary target cho các shipper. Nếu M là số target sau pruning, Hungarian assignment trên ma trận $R \times (M + R)$ có chi phí:

$$O(R^2(M + R)).$$

Với mỗi shipper, ACO chạy I vòng lặp, mỗi vòng có A ant. Mỗi ant xây route có độ sâu tối đa L , và tại mỗi bước xét tối đa K_{next} pickup candidate cùng với các delivery candidate trong túi hàng. Nếu số đơn trong túi được chặn bởi sức chứa nhỏ, chi phí route construction có thể xấp xỉ:

$$O(IALK_{\text{next}})$$

cho một shipper.

Với R shipper cần rebuild, chi phí ACO route construction là:

$$O(RIALK_{\text{next}}).$$

Do đó, độ phức tạp mỗi lần replan có thể viết gần đúng là:

$$O(RG \log G + R^2(M + R) + RIALK_{\text{next}} + C^2),$$

chưa tính các lần BFS cache miss. Trong đó $O(C^2)$ đến từ conflict resolver sau khi sinh hành động.

Trong các timestep không cần replan, solver chỉ đi theo kế hoạch cũ và xử lý xung đột, nên chi phí giảm còn khoảng:

$$O(C + C^2).$$

Về không gian, solver cần:

$$O(SN^2)$$

cho cache đường đi với S nguồn đã lưu,

$$O(R(M + R))$$

cho ma trận Hungarian,

$$O(CL)$$

cho kế hoạch hiện tại của các shipper, và

$$O(LK_{\text{next}})$$

cho pheromone cục bộ trong quá trình xây route bằng ACO. Nếu dùng pheromone toàn cục, không gian có thể tăng lên theo số transition được lưu.

3) *Mức độ tối ưu*: Thuật toán ACO trong bài toán này thuộc nhóm *metaheuristic*. Solver không đảm bảo tối ưu toàn cục, vì bài toán có tính online, đơn hàng được tiết lộ theo thời gian, tập candidate đã bị pruning, và ACO chỉ tìm kiếm trong một không gian route cục bộ với số ant và số vòng lặp hữu hạn.

Tuy nhiên, một số thành phần con có đảm bảo tối ưu trong phạm vi hẹp. BFS tìm được đường đi ngắn nhất giữa hai ô nếu bản đồ là lưới tĩnh và mọi bước di chuyển có chi phí bằng nhau. Hungarian assignment tìm được phép gán tối ưu trên ma trận lợi nhuận đã xây dựng, nhưng ma trận này chỉ chứa các target sau bước pruning nên không tối ưu trên toàn bộ tập đơn hàng. Ngoài ra, greedy baseline giúp đảm bảo route được chọn không kém hơn route tham lam theo hàm đánh giá nội bộ.

Vì vậy, có thể phân loại phương pháp này là một *online metaheuristic solver with local route improvement*. ACO có khả năng tìm route tốt hơn greedy trong các trường hợp có nhiều pickup và delivery cần sắp xếp, đặc biệt trên bản đồ lớn hoặc vùng hotspot. Tuy nhiên, chất lượng nghiệm phụ thuộc vào các tham số như số ant A , số vòng lặp I , hệ số pheromone α , hệ số heuristic β , tốc độ bay hơi pheromone ρ , độ sâu route L , và kích thước candidate K_{next} .

IV. KẾT QUẢ THỰC NGHIỆM

A. Phân tích trade-off giữa các phương pháp

Từ kết quả thực nghiệm trên các cấu hình Phase 1, có thể thấy mỗi phương pháp có ưu và nhược điểm khác nhau giữa chất lượng nghiệm, tỷ lệ giao đúng hạn và thời gian chạy.

1) *GreedyBFS*: GreedyBFS là phương pháp đơn giản nhất. Thuật toán chọn nhiệm vụ theo heuristic cục bộ và dùng BFS để di chuyển tới mục tiêu gần hoặc có điểm số tốt nhất. Ưu điểm của GreedyBFS là dễ cài đặt, dễ giải thích và không cần mô hình tối ưu phức tạp. Trên các cấu hình nhỏ như C1, C2 và C3, GreedyBFS vẫn đạt kết quả tương đối tốt, ví dụ C3 đạt net reward 870.57, gần tương đương với VRPOrToolsSolver.

Bảng I
KẾT QUẢ ĐỊNH LƯỢNG CỦA CÁC PHƯƠNG PHÁP TRÊN CÁC CẤU HÌNH PHASE 1

Config	Method	Net Reward	%Giao	%Đúng hạn	Time (s)
C1	GreedyBFS	264.57	86.7	100.0	0.1
C1	VRPOToolsSolver	267.08	86.7	100.0	0.1
C1	ACOSolver	267.06	86.7	100.0	0.1
C1	MAPDCBSSolver	264.96	86.7	100.0	0.1
C2	GreedyBFS	426.21	96.0	91.7	0.2
C2	VRPOToolsSolver	455.88	96.0	91.7	0.1
C2	ACOSolver	459.64	96.0	91.7	0.1
C2	MAPDCBSSolver	474.52	96.0	95.8	0.1
C3	GreedyBFS	870.57	92.5	91.9	0.6
C3	VRPOToolsSolver	870.41	92.5	89.2	0.3
C3	ACOSolver	905.16	92.5	97.3	0.4
C3	MAPDCBSSolver	854.52	92.5	94.6	0.5
C4	GreedyBFS	994.07	93.3	87.5	1.9
C4	VRPOToolsSolver	1021.67	93.3	91.1	0.8
C4	ACOSolver	1060.43	93.3	94.6	0.7
C4	MAPDCBSSolver	999.87	93.3	89.3	1.8
C5	GreedyBFS	1301.40	92.5	78.4	4.0
C5	VRPOToolsSolver	1427.49	93.8	85.3	1.2
C5	ACOSolver	1409.19	95.0	82.9	1.2
C5	MAPDCBSSolver	1360.93	91.2	84.9	3.7
C6	GreedyBFS	1257.63	90.0	66.7	9.2
C6	VRPOToolsSolver	1575.87	90.0	84.4	2.0
C6	ACOSolver	1572.59	91.0	82.4	1.9
C6	MAPDCBSSolver	1543.29	86.0	86.0	4.0

Tuy nhiên, nhược điểm lớn của GreedyBFS là tính myopic. Thuật toán thường chỉ nhìn mục tiêu ngắn hạn, chưa phối hợp tốt giữa nhiều shipper và chưa tối ưu chuỗi pickup-delivery dài. Điều này thể hiện rõ ở các cấu hình lớn hơn. Trên C6, GreedyBFS chỉ đạt net reward 1257.63, thấp hơn đáng kể so với VRPOToolsSolver 1575.87 và ACOSolver 1572.59. Ngoài ra, tỷ lệ đúng hạn của GreedyBFS trên C6 chỉ đạt 66.7%, cho thấy phương pháp này dễ bị trễ deadline khi số lượng đơn và kích thước bản đồ tăng. Thời gian chạy của GreedyBFS cũng tăng mạnh ở C6, lên tới 9.2s, do phải đánh giá nhiều mục tiêu bằng BFS và thiếu cơ chế pruning/tái sử dụng kế hoạch hiệu quả.

2) *VRPOToolsSolver*: VRPOToolsSolver đạt sự cân bằng tốt giữa chất lượng nghiệm và thời gian chạy. Phương pháp này sử dụng Hungarian assignment để phân công target chính cho shipper, kết hợp với candidate pruning, plan preservation và sequential multi-stop route builder. Nhờ đó, solver không chỉ chọn đơn gần nhất mà còn xét lợi ích tổng thể của việc gom đơn, giao hàng đúng hạn và giữ ổn định kế hoạch.

Kết quả cho thấy VRPOToolsSolver đạt net reward cao và ổn định trên hầu hết các cấu hình. Ở C5, phương pháp này đạt 1427.49, cao hơn GreedyBFS 1301.40 và MAPDCBSSolver 1360.93. Ở C6, VRPOToolsSolver đạt 1575.87, là kết quả cao nhất trong cấu hình này. Thời gian chạy cũng rất tốt, chỉ 2.0s ở C6, thấp hơn nhiều so với GreedyBFS 9.2s và MAPDCBSSolver 4.0s.

Trade-off của VRPOToolsSolver là thuật toán vẫn phụ thuộc nhiều vào heuristic scoring và các tham số như K , $L_{\max}$, penalty theo khoảng cách và bonus giao hàng. Nó không đảm bảo tối ưu toàn cục vì chỉ tối ưu trên tập candidate đã lọc. Tuy vậy, trong thực nghiệm Phase 1, đây là phương pháp có tính ổn định cao và phù hợp làm solver chính.

3) *ACOSolver*: ACOSolver đạt tổng điểm cao nhất trong Phase 1. Phương pháp này giữ lại pipeline mạnh của VRPOToolsSolver như candidate pruning, Hungarian assignment và plan preservation, nhưng thay phần xây dựng route bằng local ACO route improver. Thay vì chọn next target hoàn toàn tham lam, ACO cho nhiều ant thử các chuỗi pickup-delivery khác nhau, sau đó chọn route có điểm dự đoán tốt hơn greedy baseline.

Điểm mạnh của ACOSolver thể hiện rõ ở C3 và C4. Trên C3, ACOSolver đạt net reward 905.16, cao hơn VRPOToolsSolver 870.41, đồng thời đạt tỷ lệ đúng hạn 97.3%, cao nhất trong các phương pháp. Trên C4, ACOSolver đạt 1060.43, cũng là kết quả cao nhất. Tổng net reward của ACOSolver là 5674.07, cao hơn VRPOToolsSolver 5618.40.

Tuy nhiên, ACO có trade-off là tính ngẫu nhiên và phụ thuộc tham số. Kết quả có thể dao động tùy theo số ant, số vòng lặp, hệ số pheromone α , hệ số heuristic β , tốc độ bay hơi ρ và độ sâu route $L_{\max}$. Trên C5 và C6, ACOSolver có tỷ lệ giao hàng cao hơn hoặc tương đương VRPOToolsSolver, nhưng net reward hơi thấp hơn. Điều này cho thấy ACO có xu hướng khám phá nhiều route và tăng throughput, nhưng không phải lúc nào cũng tối ưu reward cuối cùng, đặc biệt nếu route được chọn làm một số đơn bị trễ nhiều hơn.

4) *MAPDCBSSolver*: MAPDCBSSolver tập trung vào xử lý xung đột giữa nhiều shipper. So với các phương pháp khác, điểm mạnh của MAPD-CBS là khả năng lập kế hoạch đường đi có xét va chạm, đặc biệt hữu ích trong các bản đồ hẹp hoặc có bottleneck. Trên C2, MAPDCBSSolver đạt net reward 474.52, cao nhất trong tất cả phương pháp, với tỷ lệ đúng hạn 95.8%. Trên C6, MAPDCBSSolver đạt tỷ lệ đúng hạn 86.0%, cao hơn ACOSolver 82.4% và VRPOToolsSolver 84.4%.

Tuy nhiên, trade-off của MAPD-CBS là chi phí tính toán

và khả năng tối ưu reward. CBS vốn phù hợp để xử lý conflict path planning, nhưng bài toán giao hàng còn cần tối ưu assignment, pickup–delivery sequence, deadline và reward. Vì vậy, nếu task assignment chưa đủ mạnh, việc path planning tốt chưa chắc tạo ra net reward cao nhất. Trên C5 và C6, MAPDCBSSolver có net reward thấp hơn VRPOToolsSolver và ACOSolver. Thời gian chạy cũng cao hơn, ví dụ C5 mất 3.7s và C6 mất 4.0s, trong khi VRPOToolsSolver chỉ mất 1.2s và 2.0s.

5) *Nhận xét tổng hợp*: Nhìn chung, GreedyBFS phù hợp làm baseline vì đơn giản và dễ giải thích, nhưng kém hiệu quả khi bài toán lớn hơn. VRPOToolsSolver là phương pháp cân bằng nhất giữa reward, deadline và runtime. ACOSolver có chất lượng nghiệm tổng thể cao nhất trong Phase 1 nhờ khả năng khám phá nhiều route cục bộ, nhưng cần kiểm soát tham số để tránh tăng độ trễ. MAPDCBSSolver có lợi thế khi conflict là vấn đề chính, nhưng có chi phí cao hơn và không nhất thiết tối đa hóa reward tốt nhất.

Vì vậy, nếu ưu tiên tốc độ và tính ổn định, VRPOToolsSolver là lựa chọn phù hợp nhất. Nếu ưu tiên tối đa hóa tổng reward và chấp nhận thêm độ phức tạp thuật toán, ACOSolver là lựa chọn tốt hơn. Nếu môi trường có nhiều nút cổ chai hoặc mật độ shipper cao gây va chạm thường xuyên, MAPDCBSSolver có thể phát huy lợi thế.

B. Chiến lược xử lý Surge và Hotspot

Trong bài toán giao hàng online, surge và hotspot là hai tình huống gây khó khăn lớn cho solver. Surge làm số lượng đơn mới tăng nhanh trong một khoảng thời gian ngắn, trong khi hotspot tạo ra các cụm pickup dày đặc tại một khu vực hẹp trên bản đồ. Nếu solver chỉ đi theo một lịch trình cố định, các shipper có thể phản ứng chậm với đơn mới, bỏ lỡ deadline hoặc di chuyển rời rạc không hiệu quả. Vì vậy, các solver chính như VRP-OrTools và ACOSolver sử dụng chiến lược phản ứng động, gồm ba cơ chế chính: cluster-aware scoring, selective real-time replanning và deadline-aware routing.

1) *Khai thác Hotspot bằng Pickup Cluster Bonus*: Để tận dụng các cụm đơn hàng xuất hiện gần nhau, solver sử dụng cơ chế *Pickup Cluster Bonus*. Khi đánh giá một đơn hàng tiềm năng g_i , solver xét các đơn hàng active khác nằm trong bán kính Manhattan $r = 4$ quanh vị trí pickup (sx_i, sy_i) . Cluster bonus được tính theo dạng:

$$B_{\text{cluster}}(g_i) = 1.0 \cdot n_i + 0.8 \cdot \sum_{g_j \in \mathcal{N}_i} p_j,$$

trong đó $\mathcal{N}_i$ là tập các đơn hàng active nằm gần g_i , $n_i = |\mathcal{N}_i|$ là số lượng đơn lân cận, và p_j là độ ưu tiên của đơn g_j .

Cơ chế này làm cho các đơn nằm trong hotspot có điểm đánh giá cao hơn so với các đơn rải rác. Khi hotspot xuất hiện, nhiều đơn hàng có pickup gần nhau sẽ cùng tạo ra cluster bonus lớn. Do đó, bước candidate pruning, Hungarian assignment và route builder có xu hướng chọn các đơn trong cụm này. Kết quả là shipper được điều hướng đến vùng có mật độ đơn cao, có khả năng gom nhiều đơn trong một chuyến nếu còn thỏa mãn ràng buộc sức chứa $K(i)$ và tải trọng $W_{\text{max}}(i)$.

2) *Phản ứng với Surge bằng Selective Real-time Replanning*: Trong điều kiện bình thường, solver không tái lập kế hoạch ở mọi timestep để tránh tốn chi phí tính toán và tránh hiện tượng shipper liên tục đổi mục tiêu. Thay vào đó, solver giữ lại kế hoạch cũ nếu kế hoạch vẫn hợp lệ. Tuy nhiên, khi có đơn mới quan trọng xuất hiện, solver có thể kích hoạt local replan ngay lập tức.

Một đơn mới g_i được xem là quan trọng nếu thỏa mãn một trong các điều kiện như có độ ưu tiên cao, deadline gần, hoặc nằm gần một shipper trong bán kính ảnh hưởng r_{affect} . Bán kính này được tính động theo kích thước bản đồ:

$$r_{\text{affect}} = \text{clip}(4 + 0.12N, 5, 18).$$

Khi phát hiện đơn mới quan trọng, biến *important_new_order* được kích hoạt và solver chuyển sang trạng thái local replan. Tuy nhiên, thay vì rebuild toàn bộ đội shipper, solver chỉ rebuild các shipper bị ảnh hưởng, ví dụ:

- shipper đang rảnh hoặc không có kế hoạch;
- shipper có kế hoạch quá ngắn;
- shipper ở gần đơn mới quan trọng;
- shipper không đang mang đơn có deadline quá gấp.

Cách làm này giúp solver phản ứng nhanh với surge nhưng vẫn tránh phá vỡ các route đang tốt. Các shipper đang mang đơn gấp thường được giữ nguyên kế hoạch để tránh làm trễ các delivery quan trọng.

3) *Deadline-aware Routing và Điều chỉnh Tham số Động*: Surge làm số lượng đơn active tăng nhanh, từ đó làm áp lực deadline tăng lên. Để xử lý tình huống này, solver sử dụng scoring có xét deadline, priority, chi phí di chuyển và delay cost. Khi xây dựng route, một pickup mới chỉ được thêm vào nếu lợi ích dự kiến đủ lớn và không gây trễ nghiêm trọng cho các đơn đang nằm trong túi hàng ảo.

Với delivery candidate, solver ưu tiên các đơn đang mang nếu deadline gần hoặc túi hàng gần đầy. Với pickup candidate, solver ước lượng cả lợi ích của việc lấy thêm đơn và tác động trễ hạn lên các đơn hiện có trong túi. Do đó, route builder có thể tạo ra các tuyến xen kẽ:

$$\text{pickup} \rightarrow \text{pickup} \rightarrow \text{delivery} \rightarrow \text{pickup} \rightarrow \text{delivery}.$$

Trong ACOSolver, ACO được dùng như một local route improver. Mỗi ant thử xây dựng một chuỗi pickup–delivery khác nhau dựa trên pheromone và heuristic desirability:

$$P(i \rightarrow j) \propto \tau_{ij}^\alpha \eta_{ij}^\beta.$$

Trong đó, τ_{ij} là pheromone của transition từ target i sang target j , còn η_{ij} được tính từ reward dự kiến, deadline, priority, cluster bonus, chi phí di chuyển và delay cost. Nhờ đó, ACO có thể thử nhiều route khác nhau trong vùng hotspot và chọn route tốt hơn greedy baseline nếu route đó có điểm dự đoán cao hơn.

Ngoài ra, solver còn có cơ chế runtime throttle. Nếu thời gian replan gần đây vượt quá budget, solver giảm dần độ sâu route L_{max} , số candidate K , và số candidate kế tiếp K_{next} . Điều này giúp solver tránh vượt giới hạn thời gian trong các đợt surge lớn.

Bảng II
THÔNG TIN NHÓM, DANH SÁCH THÀNH VIÊN VÀ PHÂN CÔNG CÔNG VIỆC

STT nhóm	Họ và tên	MSSV	Đại diện nộp bài	Công việc / Đóng góp chính
22	Bùi Phúc Bình	23021478	Có	Chạy thực nghiệm và phân tích kết quả cho các phương pháp VRP và ACO.
22	Trần Bình Minh	22028187	Không	Triển khai và hỗ trợ các phương pháp còn lại; kiểm thử thuật toán; tổng hợp kết quả thực nghiệm; hỗ trợ hoàn thiện báo cáo kỹ thuật và notebook nộp bài.

- Kaggle Notebook: <https://www.kaggle.com/code/minhtb10/graph-t-22>
- GitHub Repository: https://github.com/bui14/INT3123_1